

Cloud Firestore Enterprise edition in Native mode is now available! [Learn more.](#)

(/docs/firestore/enterprise/pipelines-overview)

Order and limit data with Cloud Firestore



Cloud Firestore provides powerful query functionality for specifying which documents you want to retrieve from a collection. These queries can also be used with either `get()` or `addSnapshotListener()`, as described in [Get Data](#) (/docs/firestore/query-data/get-data).

Note: While the code samples cover multiple languages, the text explaining the samples refers to the Web method names.

Order and limit data

By default, a query retrieves all documents that satisfy the query in ascending order by document ID. You can specify the sort order for your data using `orderBy()`, and you can limit the number of documents retrieved using `limit()`. If you specify a `limit()`, the value must be greater than or equal to zero.

Note: An `orderBy()` clause also [filters for existence of the given field](#) (`#orderby_and_existence`). The result set will not include documents that do not contain the given field.

For example, you could query for the first 3 cities alphabetically with:

Web modular API	Web namespaced API	Swift (#swift)	Objective-C (#objective-c)	Kotlin Android	Java Android
<pre>import { query, orderBy, limit } from "firebase/firestore";</pre>					

```
const q = query(citiesRef, orderBy("name"), limit(3));
90af442352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/order_and_limit.js#L8-L10
```

You could also sort in descending order to get the *last* 3 cities:

Web modular API	Web namespaced API	Swift (#swift)	Objective-C (#objective-c)	Kotlin Android	Java Android
<pre>import { query, orderBy, limit } from "firebase/firestore"; const q = query(citiesRef, orderBy("name", "desc"), limit(3)); 42352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/order_and_limit_desc.js#L8-L10</pre>					

You can also order by multiple fields. For example, if you wanted to order by state, and within each state order by population in descending order:

Web modular API	Web namespaced API	Swift (#swift)	Objective-C (#objective-c)	Kotlin Android	Java Android
<pre>import { query, orderBy } from "firebase/firestore"; const q = query(citiesRef, orderBy("state"), orderBy("population", "desc")); d90af442352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/order_multiple.js#L8-L10</pre>					

You can combine `where()` filters with `orderBy()` and `limit()`. In the following example, the queries define a population threshold, sort by population in ascending order, and return only the first few results that exceed the threshold:

Web modular API	Web namespaced API	Swift (#swift)	Objective-C (#objective-c)	Kotlin Android	Java Android
<pre>import { query, where, orderBy, limit } from "firebase/firestore"; const q = query(citiesRef, where("population", ">", 100000), orderBy("population", "asc"), limit(3)); 70af442352f058406f1aeb8adcbb/snippets/firestore-next/test-firestore/filter_and_order.js#L8-L10</pre>					

However, if you have a filter with a range comparison (<, <=, >, >=), your first ordering must be on the same field, see the list of `orderBy()` limitations below.

Limitations

Note the following restriction for `orderBy()` clauses:

- An `orderBy()` clause also filters for existence of the given fields (`#orderby_and_existence`). The result set will not include documents that do not contain the given fields.

orderBy and existence

When you order a query by a given field, the query can return only the documents where the order-by field exists.

For example, the following query would not return any documents where the **population** field is not set, even if they otherwise meet the query filters.

Java
(#java)

```
db.collection("cities").whereEqualTo("country", "USA").orderBy("population
```

A related effect applies to inequalities. A query with an inequality filter on a field also implies ordering by that field. The following query does not return documents without a **population** field even if `country = USA` in that document. As a workaround, you can execute separate queries for each ordering or you can assign a value for all fields that you order by.

Java
(#java)

```
db.collection("cities").where(or("country", USA), greaterThan("population
```

The query above includes an implied order-by on the inequality and is equivalent to the following:

Java

(#java)

```
db.collection("cities").where(or("country", USA), greaterThan("population
```

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-19 UTC.